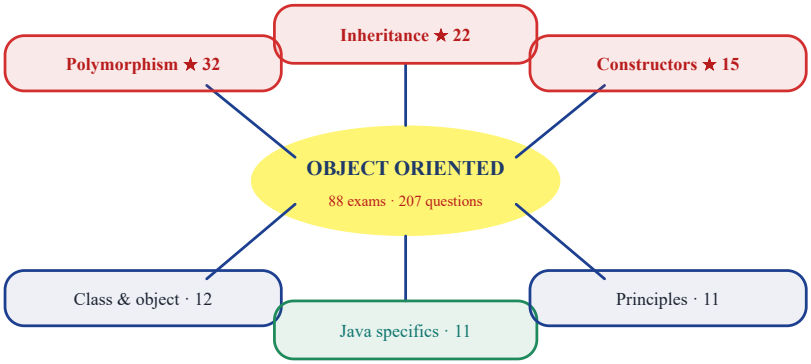
Object Oriented Programming

every OOP question asked in BCS · NTRCA · BPSC · bank · power-sector · BUET-set papers, solved in 32 sheets



↑ the number is how many different exams asked that topic (red = most asked)

How to read a question card

> **Q:** The question, as printed in the paper (typos fixed).
> ✎ **Asked in:** the exams it was taken from, e.g. BPSC AP 2023 · 49th BCS ICT (2025)

Every program on these sheets was compiled and run; green = the real output. C++ and Java are never mixed in one listing.

Most asked
① overloading vs overriding · 8
② friend function · 3
③ OOP + its pillars · 2 + 2
④ class vs object · 2
⑤ inheritance + example · 2
⑥ structured vs OOP · 2

Syllabus coverage

"**Object-oriented concepts**: class, object, **encapsulation**, **inheritance**, **polymorphism**, constructors and destructors, operator overloading, virtual functions, templates, exception handling; C++ and Java", plus threads and design patterns.

> **Legend:** **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = checked answer

Contents

✓ OOP, pillars · vs structured …… p. 2–4
✓ Class and object · class programs …… p. 5–7
✓ Encapsulation · bank programs …… p. 8–9
✓ Access specifiers · friend …… p. 10–11
✓ Inheritance I, II · programs …… p. 12–16
✓ Polymorphism · overloading …… p. 17–19
✓ Virtual · interfaces · traps … p. 20–22
✓ Constructors I–III …… p. 23–25
✓ Java, C++ · outputs …… p. 26–27
✓ Short programs …… p. 28
✓ Threads · exceptions …… p. 29–30
✓ Facade pattern · night-before … p. 31–32

1 OOP, why it is needed, the four pillars

> **Q1:** What is Object-Oriented Programming (OOP)? Why is it needed? Write its fundamental concepts (pillars). / What do you mean by OOP? Write its primary principles.
> ✎ **Asked in:** 15th NTRCA · 17th NTRCA · 15th NTRCA ICT · BOF SAE(Computer) 2021 · 37th BCS

OOP: a programming paradigm that builds a program from **objects**: each object bundles **data** (attributes) with the **methods** that work on it, and objects cooperate by calling each other's methods (**message passing**). A **class** is the blueprint the objects are made from.

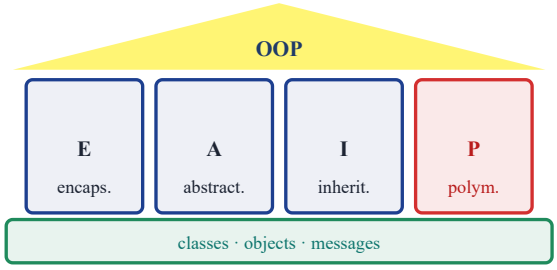
Why it is needed: in a big procedural program any function can change the shared data, so bugs spread and code is reused only by copying. OOP **protects** data inside objects, **reuses** code by inheritance, keeps a change inside **one class**, and mirrors real things (account, student), so design and maintenance are easier.

Pillar	One line to write	Example
Encapsulation	data + methods in **one unit**	`private balance` + `deposit()`
Abstraction	show **what**, hide **how**	`car.start()`, engine hidden
Inheritance	reuse and extend a class	`Savings extends Account`
Polymorphism	one interface, **many forms**	`shape.area()` per shape

Q2: On which concepts is OOP founded? Explain them. / Explain the three main principles of OOP. / What are the main features of OOP?
✎ **Asked in:** 17th NTRCA CSE (2023) · 17th NTRCA ICT (2023) · 16th NTRCA ICT · 37th BCS

Write the four pillars, then **class**, **object**, **message passing**, **dynamic binding**. For **three principles** write **encapsulation, inheritance, polymorphism** (abstraction folded into encapsulation).

MCQ as asked	answer	asked in
Object oriented programming is based on?	**objects** (and classes)	Combined Bank Officer(IT) 2026
Which is a feature of OOP? Polymorphism / Friend function / Structure / Loop	**Polymorphism**	DoICT AP 2020



Memory trick: "**A PIE**": **A**bstraction, **P**olymorphism, **I**nheritance, **E**ncapsulation. Eat the pie, write four.

Exam trap: A friend function is a C++ **feature**, not an OOP **principle**; it actually weakens encapsulation.

2 OOP concepts with one running example

Q3: Explain OOP concepts with proper examples (Java below). / Explain any two features of OOP with examples. / Explain inheritance, polymorphism and encapsulation. / What is encapsulation, inheritance and polymorphism?
✎ **Asked in:** JBL ANE 2022 · BPSC AP 2021 · 16th NTRCA CSE (2019) · 36th BCS (2016)

```
abstract class Shape {                  // abstraction: what, not how
    private String name;                // encapsulation: hidden data
    Shape(String n) { name = n; }
    String getName() { return name; }
    abstract double area();
}
class Circle extends Shape {            // inheritance: Circle IS-A Shape
    private double r;
    Circle(double r) { super("Circle"); this.r = r; }
```

```
    double area() { return Math.PI * r * r; }    // polymorphism
}
class Square extends Shape {
    private double s;
    Square(double s) { super("Square"); this.s = s; }
    double area() { return s * s; }
}
Shape[] all = { new Circle(1), new Square(2) };
for (Shape x : all) System.out.printf("%s %.2f%n", x.getName(), x.area());
```

→ Circle 3.14 · Square 4.00 (one call `x.area()`, two behaviours)

Encapsulation: `name`, `r`, `s` are private. **Abstraction**: `Shape` promises `area()` without saying how. **Inheritance**: both shapes reuse `getName()`. **Polymorphism**: the loop runs the right `area()` for each object.

> **Q4:** C++ is an object-oriented programming language: explain.
> ✎ **Asked in:** 15th NTRCA Computer Operation (2019)

C++ supports every pillar: `class` with `private/protected/public` (encapsulation), abstract classes with pure virtual functions (abstraction), five kinds of inheritance, and polymorphism by overloading (compile time) and `virtual` functions (run time). It is a **hybrid** language: C-style code with no class also compiles, so it is not "pure" OOP like Smalltalk.

> **Q5:** Write the features of object-oriented design.
> ✎ **Asked in:** WZPDCL SAE 2023

Classes model real things · **information hiding** · inheritance · polymorphism · **message passing** · modularity · reuse · **low coupling, high cohesion**.

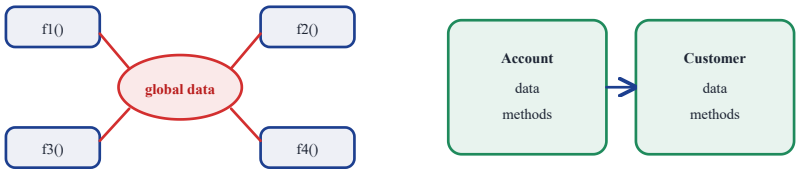
> **Q6:** Write down the principles of OOP. What is polymorphism? Write the names of 3 OOP languages.
> ✎ **Asked in:** 40th BCS (2020) · DESCO SAE 2023

Principles: the four pillars (sheet 2). Polymorphism: one name, many forms (sheet 17). Languages: **C++, Java, Python** (also C#, Smalltalk).

3 Structured (procedural) vs object-oriented

> **Q7:** Write the differences between structured programming and object-oriented programming. / Difference between object-oriented and procedural-oriented programming.
> ✎ **Asked in:** 45th BCS CSE (2024) · Combined Bank SO(AME) 2021 · Combined Bank AHE 2021

Point	Structured / procedural (POP)	Object-oriented (OOP)
unit	**function**	**object** (class)
data and code	apart; data passed around	**bound together**
data security	global data, no hiding	private data, access control
design	top-down	bottom-up
reuse	copy functions	**inheritance**, composition
overloading	not supported	function + operator overloading
communication	function calls	message passing
change	a change can ripple through	a change stays in one class
suits	small programs	large, evolving systems
examples	**C, Pascal**, FORTRAN	**C++, Java**, C#, Python



left: every function can reach the shared data · right: each object guards its own

> **Q8:** What is the difference between procedural-oriented and object-oriented programming languages? Give two examples of each.
> ✎ **Asked in:** BPSC AP 2021

Use the table, then end with exactly two each: procedural **C, Pascal**; object-oriented **C++, Java**.

> **Q9:** Write down / state the advantages of OOP over traditional structured programming.
> ✎ **Asked in:** Combined Bank Officer(IT) 2023 · 38th BCS (2018)

1. **Reuse**: inherit a tested class instead of copying code.
2. **Data security**: private data cannot be corrupted from outside.
3. **Modularity**: a team builds and tests classes separately.
4. **Easy maintenance**: a change stays inside one class.
5. **Extensibility**: add a subclass, the old code keeps working (polymorphism).
6. **Real-world modelling**: classes match the problem, so design and debugging are faster.

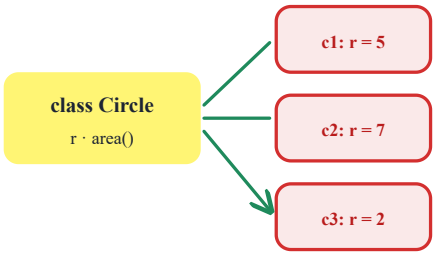
> **Memory trick:** Structured = "**verbs first**" (what to do). OOP = "**nouns first**" (who does it).

> **Exam trap:** C++ is **not** purely OOP and C is **not** OOP at all; a `struct` in C has no methods.

4 Class and object

> **Q10:** Explain the difference between class and object with examples. / What are class and object? How is an object created? / What is an object? How is it created? / Rules to declare a class and an object.
> ✎ **Asked in:** 17th NTRCA CSE (2023) · 17th NTRCA ICT (2023) · 15th NTRCA ICT · 15th NTRCA · 17th NTRCA · 16th NTRCA ICT · 15th NTRCA CSE

Class	Object
a **blueprint** / user-defined type	an **instance** of the class
logical; no memory for data	physical; occupies memory
declared once	any number created
keyword `class`	created by declaration or `new`
`class Circle`	`c1`, `c2`



one class, many objects

```
class Circle {                    // declare: keyword class, name, { members };
    double r;                     // data member (private by default)
public:
    Circle(double radius) { r = radius; }
    double area() { return 3.1416 * r * r; }    // member function
};
int main() {
    Circle c1(5);                 // object on the stack
    Circle *c2 = new Circle(7);   // object on the heap (Java: always new)
    cout << c1.area() << " " << c2->area();    delete c2;
}
```

→ 78.54 153.938

Q11: What are objects in C++? / How is a member function of a class defined? / What is a method in OOP, and how is it used?

✎ **Asked in:** 37th BCS · 16th NTRCA ICT

An **object** is a variable of a class type: it has **state** (its data members), **behaviour** (member functions) and **identity** (its own address). A **member function (method)** is a function declared inside the class; it is called on an object (`c1.area()`) and works on that object's data. Define it inside the class (then it is implicitly `inline`) or outside with the **scope operator ::** — `double Circle::area() { return 3.1416*r*r; }`.

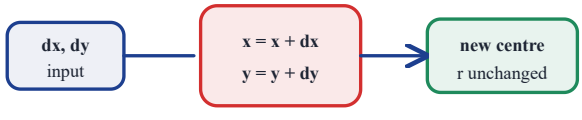
MCQ The class in C++ contains: (a) data only (b) function only (c) data and function both (d) none → **(c)** ✎ 49th BCS ICT (2025)

5 Designing a class: data, methods, objects

Q12: We are developing software for a graphics package and must implement a circle class. The circle must be translatable from its origin and must give its perimeter and area. Identify the data and method requirements and give the data flow of the translation method.

✎ **Asked in:** Combined Bank Officer(IT) 2024

Data (private)	Methods (public)
`x, y`: centre	`translate(dx, dy)`
`r`: radius	`perimeter()` = 2πr
	`area()` = πr²
	constructor, `centre()`



perimeter and area do not change

data flow of translate()

```
class Circle {                                  // Java
    private double x, y, r;
    Circle(double x, double y, double r) { this.x = x; this.y = y; this.r = r; }
```

```
    void translate(double dx, double dy) { x += dx; y += dy; }
    double perimeter() { return 2 * Math.PI * r; }
    double area() { return Math.PI * r * r; }
}
Circle c = new Circle(0, 0, 2);   c.translate(3, 4);
```

→ centre (3.0, 4.0) · perimeter 12.57 · area 12.57

Q13: `Class test(){ Int Public: t(int value = 0){} t.getone(value) t.gettwo(value+10)} Int main(){ Test (5); cout<<"Get One"<<getone<<endl; cout<<"Get One"<<gettwo<<endl; }` (write it correctly)

✎ **Asked in:** Dhaka WASA AME(Network) 2025

```
class Test {
    int value;
public:
    Test(int v = 0) { value = v; }
    int getOne() { return value; }
    int getTwo() { return value + 10; }
};
int main() {
    Test t(5);
    cout << "Get One " << t.getOne() << endl;
    cout << "Get Two " << t.getTwo() << endl;
}
```

Exam trap: As printed it cannot compile: `Class`/`Int`/`Public` are capitalised, the class needs a name not `test()`, the constructor must have the class name, members are called on an object (`t.getOne()`).

→ Get One 5 · Get Two 15

6 Class programs and C++ defaults

Q14: Write a C++ program with a class `student` having name, roll and marks, use get methods, create two objects and print the name of the student with the higher marks.

✎ **Asked in:** Cadet College Lecturer(ICT) 2025

```
class Student {
    string name; int roll; double marks;
public:
    Student(string n, int r, double m) { name = n; roll = r; marks = m; }
    string getName() { return name; }   double getMarks() { return marks; }
};
int main() {
    Student s1("Nusrat", 1, 78.5), s2("Tanvir", 2, 84);
    Student top = (s1.getMarks() > s2.getMarks()) ? s1 : s2;
    cout << "Highest: " << top.getName() << " (" << top.getMarks() << ")";
}
```

→ Highest: Tanvir (84)

Q15: Write down the difference between structure and class.

✎ **Asked in:** BCIC AP 2025

In C++ both can hold data and functions; the differences are defaults: `struct` members and inheritance are **public** by default, `class` members and inheritance are **private**. `struct` is used for plain data, `class` for objects with hidden data. (In C a `struct` has no functions; in C# a `struct` is a value type.)

Q16: Creating an object of an inner class.
✎ **Asked in:** BUET M.Sc. May 2019

```
class Outer {                                  // Java
    int x = 5;
    class Inner { int twice() { return x * 2; } }     // inner
    static class Nested { int one() { return 1; } }   // static nested
}
Outer o = new Outer();
Outer.Inner in = o.new Inner();       // needs an Outer object
Outer.Nested n = new Outer.Nested();  // does not
System.out.println(in.twice() + " " + n.one());
```

→ 10 1

A (non-static) **inner class** object lives inside an outer object and can read its fields, so it is made with **outer.new Inner()**. A **static nested** class needs no outer object.

Q17: C++-related MCQs on protected, private and inheritance (the five questions were not preserved).
✎ **Asked in:** BUET M.Sc. April 2017

They test the **inheritance mode** table: a base member's access in the derived class.

base member →	public	protected	private
`: public Base`	public	protected	not accessible
`: protected Base`	protected	protected	not accessible
`: private Base`	private	private	not accessible

7 Encapsulation and abstraction

Q18: Briefly describe abstraction and encapsulation.
✎ **Asked in:** BCC Programmer 2019

	Encapsulation	Abstraction
meaning	wrap data + methods in **one unit** and restrict access	show only the **essential** features, hide the details
hides	the **data** (state)	the **implementation** (complexity)
level	implementation	design
tools	`private` + getters/setters	abstract class, interface
example	`balance` changed only by `deposit()`	ATM "withdraw" button

Q19: How can we solve a problem using abstraction?
✎ **Asked in:** 37th BCS

① list what the user must do, ignore how · ② write it as an **interface** / **abstract class**: `interface Payment { void pay(double amt); }` · ③ hide each way in its own class (`BKash`, `Card` implement `pay()`) · ④ the program uses only `Payment`, so adding Nagad changes nothing else.

Q20: Explain encapsulation and inheritance in detail. / Explain how encapsulation and inheritance are advantageous in OOP.
✎ **Asked in:** 38th BCS (2018) · Combined Bank Officer(IT) 2024

Encapsulation binds data to the methods that use it and hides it with `private`; outsiders use checked public methods. Gains: **data integrity**, inside can change freely, simpler use. **Inheritance** lets a derived class acquire the members of a base class and add or override its own (IS-A). Gains: **code reuse**, less testing, a natural hierarchy, run-time polymorphism.

MCQ as asked	answer	asked in
Binding data and code together?	**Encapsulation**	49th BCS ICT (2025)
___ is used to restrict access to certain details of an object	**Encapsulation**	49th BCS CSE (2025)
Hiding the internal implementation details of an object (no "abstraction" option)	**Encapsulation**	Combined Bank ADA 2021
Which does NOT achieve encapsulation? private / classes / getters-setters / global variables	**global variables**	BB AD(ICT) 2025

Memory trick: **Abstraction** = the **menu** (what you can order). **Encapsulation** = the **kitchen door** (you cannot walk in).

Exam trap: "Restrict **access**" → encapsulation; "show only **essential** features" → abstraction. Read the verb.

8 Two bank-account programs

Q21: Write a program in any object-oriented language (C++/Java/Python) to represent a bank account: a class `BankAccount` with data members for account holder name, account number and balance; member functions to deposit money, withdraw money (ensuring sufficient balance) and display account details. Demonstrate encapsulation by keeping the data members private and providing public methods to access and modify them. 10 marks

✎ **Asked in:** Combined Bank Officer(IT) 2025

```
class BankAccount {                                          // Java
    private String holder, accountNo;  private double balance;   // hidden
    public BankAccount(String holder, String accountNo, double balance) {
        this.holder = holder; this.accountNo = accountNo; this.balance = balance;
    }
    public void deposit(double amt) { if (amt > 0) balance += amt; }
    public boolean withdraw(double amt) {
        if (amt > 0 && amt <= balance) { balance -= amt; return true; }
        System.out.println("Insufficient balance"); return false;
    }
    public double getBalance() { return balance; }
    public void display() { System.out.println(accountNo + " " + holder + " " + balance); }
}
// in main():
BankAccount a = new BankAccount("Karim", "0101-77", 5000);
a.deposit(1500);  a.withdraw(2000);  a.withdraw(9000);  a.display();
```

→ Insufficient balance · 0101-77 Karim 4500.0

Marks: fields **private** · `withdraw` checks `amt <= balance` · a getter but no setter for `balance` · `a.balance = -500;` does not compile.

Q22: Make a Python class named `BankAccount`. It has a name and a balance, and supports three operations: 1. deposit 2. withdraw 3. show current balance.

✎ **Asked in:** BUET AP 2025

```
class BankAccount:
    def __init__(self, name, balance=0):
```

→ Insufficient balance
→ Rahim: 1200

```
        self.name = name
        self.__balance = balance      # private (name mangling)
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient balance")
    def show_balance(self):
        print(f"{self.name}: {self.__balance}")
acc = BankAccount("Rahim", 1000)
acc.deposit(500);  acc.withdraw(300)
acc.withdraw(5000); acc.show_balance()
```

`__init__` is Python's **constructor**; `self` is the object (like `this`). Two leading underscores make `__balance` private by **name mangling**: `acc.__balance` raises `AttributeError`.

> **Memory trick:** Every bank question is the same **three-check** answer: amount > 0 · amount ≤ balance · data private.

9 Access specifiers: public, protected, private

> **Q23:** What are the various access specifiers in C++? Explain their purpose with an example.
> ✎ **Asked in:** BPSC ADA 2022

An **access specifier** decides who may use a class member. `public`: everyone · `protected`: the class, its friends and its **derived classes** · `private`: the class and its friends only (the default in a C++ `class`).

```
class Account {
private:   double balance = 0;        // only Account (and its friends)
protected: double rate = 0.05;        // + classes derived from Account
public:    void deposit(double t) { if (t > 0) balance += t; }   // everyone
           double get() { return balance; }
};
class Savings : public Account {
public: double yearly() { return get() * rate; }   // rate: OK, balance: error
};
Savings s;  s.deposit(1000);  cout << s.get() << " " << s.yearly();
```

→ 1000 50

> **Q24:** Java has three access specifiers. Find which work with Public, Private and Protected mode; write Y or N. Columns: within class · within package · outside package by subclass only · outside package.
> ✎ **Asked in:** Petrobangla AM(CSE) 2024

Specifier	Within class	Within package	Outside package, subclass	Outside package
`public`	**Y**	**Y**	**Y**	**Y**
`protected`	**Y**	**Y**	**Y**	**N**
`private`	**Y**	**N**	**N**	**N**
(default, no keyword)	Y	Y	N	N

> **Exam trap:** One printed solution is an **identity matrix** (Y only for the same-named "mode"); it says nothing. Answer: **public YYYY · protected YYYN · private YNNN**.

MCQ as asked	answer	asked in
Which modifier cannot be applied to a method in C++? Protected / Private / Public / Abstract	**Abstract**	BPSC AP 2022
Default inheritance in C++? Public / Private / Protected / Compile-time error	**Private**	BPSC AME 2019

`abstract` is not a C++ keyword (the compiler says "unknown type name 'abstract'"); C++ makes a method abstract with `= 0`. `class B : A {}` is **private** inheritance, `struct B : A {}` is public.

10 Friend function

Q25: What is a friend function? Write its characteristics, advantages and disadvantages.
✎ **Asked in:** 16th NTRCA · 16th NTRCA ICT · 18th NTRCA College ICT (2024)

Friend function: a non-member function that a class names with the keyword `friend` inside its body; it may then access that class's **private and protected** members.

Characteristics

- ✓ declared inside the class, **defined outside** without `friend` or `::`
- ✓ **not a member**: no `this`, not in the class's scope
- ✓ called like a normal function: `show(b)`, not `b.show()`
- ✓ gets the object as an **argument** and uses `obj.member`
- ✓ friendship is **not inherited**, not mutual, not transitive

Advantages

- ✓ one function can work on the private data of **two classes**
- ✓ needed for `operator<<` / `>>` on objects
- ✓ avoids many getters

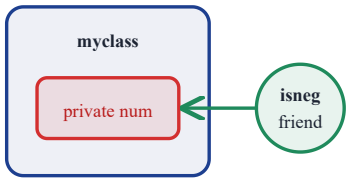
Disadvantages

- ✓ breaks **encapsulation**
- ✓ bypasses the class's own checks
- ✓ more coupling, harder maintenance

Q26: What is a friend function? Given the class below, show how to add a friend function named `isneg()` that takes one parameter of type `myclass` and returns true if `num` is negative and false otherwise. `class myclass { int num; public: myclass(int i) { num = i; } };`
✎ **Asked in:** Hydrocarbon Unit AD(ICT) 2025

```
class myclass {
    int num;
public:
    myclass(int i) { num = i; }
    friend bool isneg(myclass ob);   // declared inside
};
bool isneg(myclass ob) {             // defined outside,
    return ob.num < 0;               // reads private num
}
int main() {
    myclass a(-5), b(8);
    cout << boolalpha << isneg(a) << " " << isneg(b);
}
```

→ true false



outside the class, inside the wall

Friend class: `friend class B;` inside `A` lets **every** member function of `B` use A's private members; a friend function opens the door to **one** function.

Memory trick: A friend has the **house key** but is not **family**: no `this`, no inheritance.

Exam trap: Writing `a.isneg()` or `bool myclass::isneg` loses the marks; a friend is not a member.

11 Inheritance: definition, types, pros and cons

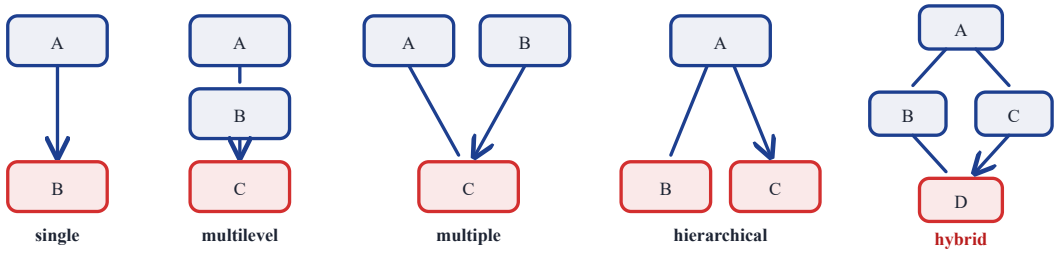
Q27: What is inheritance? Explain with an example. / Discuss its types, advantages and disadvantages. / Explain the concept of inheritance; name and briefly describe three common types.

✎ **Asked in:** 17th NTRCA CSE (2023) · 17th NTRCA ICT (2023) · 17th NTRCA · Combined Bank AP 2026 · Bank (name cut off) AP

Q28: Inheritance is one of the important issues for any object-oriented programming language. Its main advantage is the ability to reuse code. Explain briefly the different types of inheritance.

✎ **Asked in:** Combined Bank Officer(IT) 2020

Inheritance: creating a new class (**derived / child**) from an existing class (**base / parent**), so the derived class gets the base members and adds or overrides its own. It models an **IS-A** relation: a Savings account *is an* Account.



the five types (arrow = "is inherited by"); the three common ones are single, multilevel, hierarchical

```
class A {};
class B : public A {};              // single:       A → B
class C : public A {};              // hierarchical: A → B and A → C
class D : public C {};              // multilevel:   A → C → D
class E : public B, public C {};    // multiple:     B + C → E   (Java: only via interfaces)
class F : public D, public E {};    // hybrid:       two kinds at once
```

Advantages

- ✓ **code reuse**: tested code is not rewritten
- ✓ less code, less testing
- ✓ natural hierarchy, easy to extend
- ✓ enables run-time polymorphism

Disadvantages

- ✓ **tight coupling**: a base change breaks children
- ✓ deep hierarchies are hard to follow
- ✓ multiple inheritance → **diamond problem**
- ✓ child carries unused base members

Q29: How many classes can be used in hybrid inheritance?

✎ **Asked in:** BCC AP 2023

Assumption stated: there is **no upper limit**. The minimum depends on the two kinds combined: A → B plus (A, B) → C needs only 3, and the textbook diamond above (hierarchical + multiple) uses **4**. Write both numbers with the drawing.

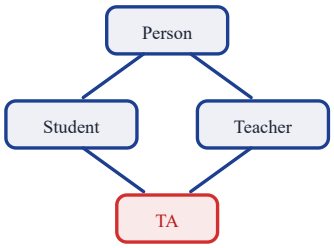
12 Diamond, constructor order, overriding

Q30: What is the diamond problem of multiple inheritance in C++?

✎ **Asked in:** BPSC AP 2023

When `Student` and `Teacher` both inherit `Person`, and `TA` inherits both, a `TA` object holds **two copies** of `Person`. `t.id` is then **ambiguous**: the compiler says "non-static member 'id' found in multiple base-class subobjects". Fix: **virtual inheritance**, which keeps one shared `Person`.

```
class Person { public: int id = 7;
               Person() { cout << "Person "; } };
class Student : virtual public Person {
    public: Student() { cout << "Student "; } };
class Teacher : virtual public Person {
    public: Teacher() { cout << "Teacher "; } };
class TA : public Student, public Teacher {
    public: TA() { cout << "TA\n"; } };
int main() { TA t;  cout << t.id; }
```



→ Person Student Teacher TA
→ 7 (one Person only)

MCQ Class C inherits B, and B inherits A. Creating an object of C calls constructors in which order? (a) C, B, A (b) A, C, B (c) A, B, C (d) C, A, B → **(c) A, then B, then C**; destructors run in reverse (~C ~B ~A), checked by running it. ✎ BUET IICT April 2021

Q31: Write the definitions of inheritance and polymorphism with a coding example. / Explain inheritance and polymorphism in Java; write a Java program to demonstrate method overriding.
✎ **Asked in:** Combined Bank AP 2021 · Combined Bank Officer(IT) 2026

```
class Animal {
    void sound() { System.out.println("Animal makes a sound"); }
}
class Dog extends Animal {                  // inheritance
    @Override void sound() { System.out.println("Dog barks"); }
}
class Cat extends Animal {
    @Override void sound() { System.out.println("Cat meows"); }
}
// main():
Animal[] a = { new Dog(), new Cat(), new Animal() };
for (Animal x : a) x.sound();               // polymorphism
```

→ Dog barks
→ Cat meows
→ Animal makes a sound

Inheritance: `Dog` gets everything of `Animal`. **Polymorphism**: an `Animal` reference runs the **object's** own `sound()`, decided at run time. `@Override` makes the compiler check the signature.

Memory trick: Constructors build a house **from the foundation up** (A → B → C); destructors pull it down **from the roof**.

Exam trap: Java has **no** multiple inheritance of classes, so no diamond of classes; a class may implement many interfaces.

13 Inheritance programs: `super` at work

Q32: Complete the Java program: class `A` has `alpha`, `beta` and `display()`; class `Gamma` extends `A` and adds `gamma`. Write its constructor, override `display()`, and write a `main` that creates `Gamma(3, 30, 10)`.
✎ **Asked in:** BCC AP 2021

```
class A {
    int alpha, beta;
    A(int alpha, int beta) { this.alpha = alpha; this.beta = beta; }
    void display() { System.out.println("Alpha" + alpha);
                     System.out.println("Beta" + beta); }
}
class Gamma extends A {
```

→ Alpha3
→ Beta30
→ Gamma10

```
    int gamma;
    Gamma(int alpha, int beta, int gamma) {
        super(alpha, beta);             // base part first
        this.gamma = gamma;
    }
    @Override
    void display() { super.display(); System.out.println("Gamma" + gamma); }
}
public class Main {
    public static void main(String[] args) { new Gamma(3, 30, 10).display(); }
}
```

Exam trap: The printed solution does not compile: it writes `@override` (must be **@Override**) and `public class main(` (must be {).

`super(...)` must be the **first** statement of the constructor; `super.display()` reuses the parent's printing.

Q33: Consider the Java class `Rectangle` (private `length`, `width`, `height`; a method that sets length and width; `getArea()` returns length*width). Make a class `Box` that inherits `Rectangle` and uses the same methods; do not declare extra methods or variables.

✎ **Asked in:** BAERA Engineer(CSE) January 2021

```
class Rectangle {
    private double length, width;
    public void setValue(double length, double width) {
        this.length = length; this.width = width; }
    public double getArea() { return length * width; }
}
class Box extends Rectangle { }      // nothing new: all inherited
// main():
Box b = new Box();  b.setValue(4, 2.5);
System.out.println(b.getArea());
```

→ 10.0

The printed `getvalue` is declared `double` but returns nothing, so it is written as `void setValue`. `Box` cannot touch the **private** fields directly; it reaches them through the inherited **public methods**.

14 Base + derived class programs

Q34: Develop a program using OOP principles. (a) 4 A base class `Employee` with private/protected Employee ID, Employee Name and Basic Salary, a constructor, and a method to display the information. (b) 5 A derived class `PermanentEmployee` with methods to calculate a 20% allowance on basic, the total salary, and to display the total with the information. (c) 4 In `main()` create three `PermanentEmployee` objects, display each one's details and total, and display the employee with the highest total.

✎ **Asked in:** ICB Securities AP 2026(Aptitude)

```
class Employee {
    protected int id;  protected String name;  protected double basic;
    Employee(int id, String name, double basic) { this.id = id; this.name = name; this.basic = basic; }
    void display() { System.out.println(id + " " + name + " basic " + basic); }
}
class PermanentEmployee extends Employee {
    PermanentEmployee(int id, String name, double basic) { super(id, name, basic); }
    double allowance() { return basic * 0.20; }
    double total() { return basic + allowance(); }
    void displayTotal() { display(); System.out.println("  total " + total()); }
}
```

```
public class Main {
    public static void main(String[] args) {
        PermanentEmployee[] e = { new PermanentEmployee(101, "Asif", 40000),
            new PermanentEmployee(102, "Mitu", 52000), new PermanentEmployee(103, "Rony", 45000) };
        PermanentEmployee top = e[0];
        for (PermanentEmployee p : e) { p.displayTotal(); if (p.total() > top.total()) top = p; }
        System.out.println("Highest: " + top.name + " " + top.total());
    }
}
```

→ 101 Asif basic 40000.0 / total 48000.0 · 102 Mitu … total 62400.0 · 103 Rony … total 54000.0 · **Highest: Mitu 62400.0**

Q35: Write a class `Bicycle` with a constructor that initialises the integer members `cadence`, `gear` and `speed`. Write another class `MountainBike` that extends `Bicycle`, has its own constructor, adds the integer field `seatHeight` and a method to set it.

✎ **Asked in:** DESCO AE(CSE) 2016

```
class Bicycle {
    int cadence, gear, speed;
    Bicycle(int cadence, int gear, int speed) {
        this.cadence = cadence; this.gear = gear; this.speed = speed; }
}
class MountainBike extends Bicycle {
    int seatHeight;
    MountainBike(int cadence, int gear, int speed, int seatHeight) {
        super(cadence, gear, speed);
        this.seatHeight = seatHeight; }
    void setHeight(int h) { seatHeight = h; }
}
MountainBike m = new MountainBike(60, 3, 20, 28);  m.setHeight(30);
```

→ 60 3 20 30

Memory trick: Child constructor = "**call parent, then mine**": `super(...)` first, own fields after.

15 A class hierarchy with virtual functions

Q36: Write a C/C++ program that has a class `Account` and subclasses `SavingsAccount`, `CurrentAccount` etc. in a proper hierarchy.

✎ **Asked in:** Combined Bank AP 2021

```
class Account {
protected:  string owner;  double balance;
public:
    Account(string o, double b) : owner(o), balance(b) {}
    void deposit(double amt) { if (amt > 0) balance += amt; }
    virtual bool withdraw(double amt) {
        if (amt > 0 && amt <= balance) { balance -= amt; return true; }  return false; }
    virtual void monthEnd() {}
    void show() { cout << owner << ": " << balance << "\n"; }
    virtual ~Account() {}
};
class Savings : public Account {                // earns interest
    double rate;
public:
    Savings(string o, double b, double r) : Account(o, b), rate(r) {}
    void monthEnd() override { balance += balance * rate / 12; }
};
class Current : public Account {                // may overdraw up to a limit
```

```
    double overdraft;
public:
    Current(string o, double b, double od) : Account(o, b), overdraft(od) {}
    bool withdraw(double amt) override {
        if (amt > 0 && amt <= balance + overdraft) { balance -= amt; return true; }  return false; }
};
int main() {
    Account* acc[2] = { new Savings("Rina", 12000, 0.06), new Current("Babul", 5000, 3000) };
    acc[0]->withdraw(2000);  acc[0]->monthEnd();          // 10000 + 50 interest
    acc[1]->withdraw(7000);                               // allowed: within the overdraft
    cout << boolalpha << acc[1]->withdraw(2000) << "\n";  // -4000 > limit: refused
    for (Account* a : acc) { a->show(); delete a; }
}
```

→ false · Rina: 10050 · Babul: -2000

Exam trap: Without `virtual` on `withdraw`, `acc[1]->withdraw()` would run the **base** rule and refuse the overdraft. Give the base class a **virtual destructor** when you `delete` through a base pointer.

Any "base + derived" program in five steps

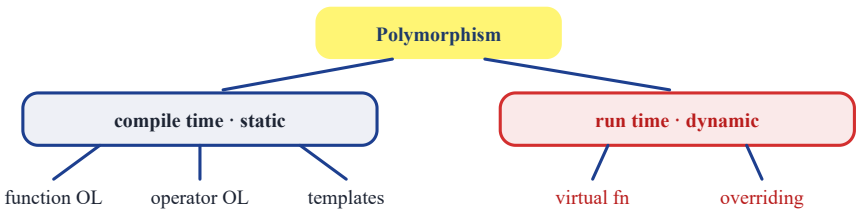
1. Base class: **common** data (`protected` or `private` + getters) and a constructor.
2. Common methods in the base; methods that differ are `virtual` (C++) / overridable (Java).
3. Each derived class: its **extra** data, a constructor that calls the base one first (`: Base(...)` / `super(...)`).
4. Override only what differs (`override` / `@Override`).
5. `main`: base-type array or pointers to mixed objects, one loop calling the same method.

16 Polymorphism and its two kinds

Q37: What is polymorphism? Discuss its different types with examples. / What do you understand by polymorphism? Describe with an example. / Explain the concept of polymorphism in OOP with an example. / What is polymorphism? Explain in the light of Java.

✎ **Asked in:** Combined Bank AP 2024 · Parliament Secretariat Programmer 2023 · MoFood Network/Website Manager 2025 · BTCL JAM 2022

Polymorphism: ("many forms") one name — function, operator or reference — behaves differently by the **argument types** or the **actual object**: `+` adds numbers and joins strings; `shape.area()` per shape.



OL = overloading

	Compile-time	Run-time
decided by	compiler, from argument types	JVM / vtable, from the **object**
achieved by	**overloading**, templates	**overriding** + base reference
in Java	overloading only (no operator overloading)	overriding; methods virtual by default

Q38: What is polymorphism? Write a Java program to demonstrate compile-time and run-time polymorphism.
✎ **Asked in:** 46th BCS CSE

```
class Calc {                                  // compile time
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}
// Animal, Dog, Cat with sound() overridden: sheet 13
Calc c = new Calc();
System.out.println(c.add(2, 3) + " " + c.add(2.5, 1.5) + " " + c.add(1, 2, 3));
Animal[] a = { new Dog(), new Cat(), new Animal() };
for (Animal x : a) x.sound();                 // run time
```

→ 5 4.0 6 · Dog barks · Cat meows · Animal makes a sound

MCQ Which one is compile-time polymorphism? (a) virtual function (b) operator overloading (c) function overriding (d) dynamic binding
→ **(b)** ✎ 49th BCS CSE (2025)

17 Method overloading vs overriding

Q39: Explain / define method overloading and method overriding with examples.
✎ **Asked in:** 17th NTRCA ICT (2023) · 36th BCS (2016) · BPSC AME 2021 · BPSC AP 2023 · Combined Bank AHE 2021 · Combined Bank SO(AME) 2021 · NU AP 2020 · RAKUB Programmer 2021

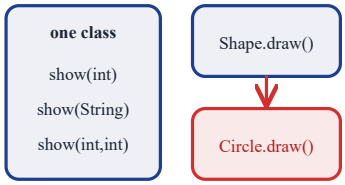
Q40: What is the difference between method overloading and method overriding? Describe with an example. / Explain polymorphism, and the method of overloading and of overriding, with examples. / Differentiate function overloading and function overriding with examples.
✎ **Asked in:** Parliament Secretariat Programmer 2023 · MoFood Network/Website Manager 2025 · 15th NTRCA · 15th NTRCA ICT · 46th BCS ICT

Point	Overloading	Overriding
what	same name, **different parameters**	same name, **same signature**, new body
where	usually in **one class**	in a **subclass** (needs inheritance)
resolved	**compile time** (static)	**run time** (dynamic)
return type	may differ (but not alone)	same (or a subtype in Java)
access	any	cannot be made weaker
static / private / final	can be overloaded	cannot be overridden
C++ keyword	none	`virtual` in the base
purpose	convenience: one name for one idea	specialise inherited behaviour

```
class Printer {                               // overloading
    void show(int n) { System.out.println("int " + n); }
    void show(String s) { System.out.println("String " + s); }
    void show(int a, int b) { System.out.println("two " + (a + b)); }
}
class Shape { void draw() { System.out.println("Shape"); } }
class Circle extends Shape {                  // overriding
    @Override void draw() { System.out.println("Circle"); }
}
```

→ int 5
→ String hi
→ two 5
→ Circle

```
// main():
Printer p = new Printer();
p.show(5);  p.show("hi");  p.show(2, 3);       // compile time
Shape s = new Circle();
s.draw();                                      // run time
```



side by side vs top to bottom

Memory trick: Over**load** = put more **loads** (parameter lists) on one name, side by side. Over**ride** = the child **rides over** the parent's method, top to bottom.

Exam trap: Changing **only the return type** is neither: it is a compile error ("method already defined").

18 Function and operator overloading

Q41: What is function overloading? Explain with an example. / What are function overloading and operator overloading? Give examples. / What is compile-time polymorphism, and how is it achieved (overloading)? Give a code example.

✎ **Asked in:** 18th NTRCA College ICT (2024) · 40th BCS (2020) · 16th NTRCA CSE (2019) · BUET M.Sc.

Function overloading: several functions share one name but differ in the **number, type or order** of parameters; the compiler picks one from the call. **Operator overloading**: giving an operator (`+`, `==`, `<<`) a meaning for a class by writing `operator+`. Both are **compile-time polymorphism**.

```
int area(int s) { return s * s; }                 // square
int area(int l, int w) { return l * w; }          // rectangle
double area(double r) { return 3.1416 * r * r; }  // circle
class Complex {
    double re, im;
public:
    Complex(double r = 0, double i = 0) { re = r; im = i; }
    Complex operator+(const Complex &c) { return Complex(re + c.re, im + c.im); }
    friend ostream& operator<<(ostream &os, const Complex &c)
        { return os << c.re << "+" << c.im << "i"; }
};
cout << area(4) << " " << area(3, 5) << " " << area(1.0);
Complex a(1, 2), b(3, 4);   cout << a + b;
```

→ 16 15 3.1416
→ 4+6i

MCQ A function having more than one distinct meaning is called a ___ function: parameter / prototype / overloaded / polymorphism → **overloaded** ✎ BPSC AP 2022

Q42: Why are the overloaded functions `int f(int a, int b = 5);` and `int f(int &a);` inherently ambiguous?

✎ **Asked in:** 38th BCS (2018)

For `int x; f(x);` both match equally well: the first through its **default argument** (`b` becomes 5), the second by binding the reference to `x`. Passing by value and by reference look identical at the call, so neither is better and the compiler stops: error: call to 'f' is ambiguous.

Exam trap: The printed solution says `int b = 5` "should not be in the question". Wrong: the default argument is exactly **why** a one-argument call can reach the first function.

MCQ as asked	answer	asked in
Overload as a global function, not a member? postfix ++ / comparison / "arithmetic" << / prefix ++	<< (insertion)	BB AD(ICT) 2025
X has `operator*(X)`, `operator*(int)`, `operator+(X)`. Which compiles? (a) ob2 + 2*ob3 (b) ob2*2 + ob3 (c) (ob2 + 2)*ob3	**(b)**	BUET M.Sc. April 2021

`cout << obj`: `ostream` is on the left, so `<<` cannot be our member. (a) needs `int*X`, (c) `X+int`: neither exists (compiler-checked).

19 Static vs dynamic binding, virtual functions

Q43: What are static binding and dynamic binding in OOP? Explain. / Explain early binding and late binding with an example.

✎ **Asked in:** BPSC AP 2021 · 36th BCS (2016)

	Static (early) binding	Dynamic (late) binding
call linked to body	at **compile time**	at **run time**
decided by	type of the **pointer / reference**	type of the **object** it points to
used for	normal, overloaded, static, private functions	`virtual` functions (C++), overridden methods (Java)

```
class Shape { public:
    virtual void draw() { cout << "Shape\n"; }
    void name() { cout << "shape\n"; }
    virtual ~Shape() {} };
class Circle : public Shape { public:
    void draw() override { cout << "Circle\n"; }
    void name() { cout << "circle\n"; } };
Shape* p = new Circle;
p->draw();     // virtual: late binding
p->name();     // not virtual: early binding
```

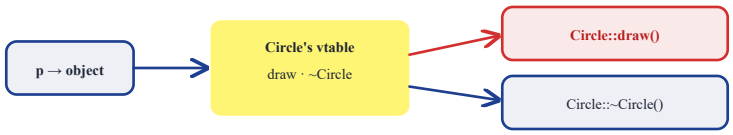
→ Circle
→ shape

Same pointer, two answers: only the `virtual` call looks at the real object.

Q44: What is a virtual function? Explain with an example.

✎ **Asked in:** BITAC AP 2023

A **virtual function** is a base-class member function declared with `virtual` and redefined in derived classes; a call through a base pointer or reference runs the version of the **actual object** (example above). Rules: non-static member, same signature in the derived class, virtual destructor in the base. A **pure virtual function** has no body (`= 0`), which makes the class **abstract**: `Shape s;` fails with "variable type 'Shape' is an abstract class".



late binding: object → its class's vtable → the right body

MCQ as asked	answer	asked in
Which declares a pure virtual function in C++? `virtual void f() = 0;` / `virtual void f();` / `void f() = 0;` / `pure virtual f();`	`virtual void f() = 0;`	49th BCS ICT (2025)
In Java, which keyword prevents a method from being overridden? const / final / static / synchronized	**final**	49th BCS ICT (2025)

20 Interfaces, and the override / hide puzzle

Q45: What is the use of an interface in object-oriented programming?
✎ **Asked in:** 16th NTRCA ICT

An **interface** is a pure contract: method signatures with no state. Uses: ① **abstraction** — callers depend on *what*, not *how* · ② a class may implement **many interfaces**, Java's safe form of multiple inheritance · ③ **loose coupling**: swap implementations (`List` → `ArrayList` / `LinkedList`) · ④ run-time polymorphism through an interface reference.

Q46: Implement the abstract method of the interface: write the child classes so that this `main` runs. `interface Fly { void print(); }` … `Fly dog = new Dog(); Fly bird = new Bird(); dog.print(); bird.print();`
✎ **Asked in:** BCIC AP 2025

```
class Dog implements Fly {
    public void print() { System.out.println("Dog cannot fly"); } }
class Bird implements Fly {
    public void print() { System.out.println("Bird can fly"); } }
```

→ Dog cannot fly
→ Bird can fly

The paper's expected output was not preserved, so print any two lines. Interface methods are implicitly `public`: dropping `public` is a compile error.

Q47: Determine the overloaded, overridden and hidden superclass methods: `class A { public void m1() {} public void m2() {} public void m3(int i) {} public void m4(String i) {} } class B extends A { public static void m1() {} public void m2() {} public void m3(int i) {} public static void m4(String i) {} }`
✎ **Asked in:** BB AP 2023

Overridden: `m2()`, `m3(int)` (same signature, instance methods). **Overloaded:** none (no same name with other parameters). **Hidden:** as printed, **none — it does not compile**: javac says "m1() in B cannot override m1() in A" because a static method cannot hide an **instance** method. Hiding happens only when the parent's method is static too:

```
class A { public static void m1() { System.out.println("A.m1"); } ... }
class B extends A { public static void m1() { System.out.println("B.m1"); } ... }
A ref = new B();
ref.m1(); ref.m2(); ref.m3(1); ref.m4("x");    // m1, m4 static in both
```

→ A.m1 · B.m2 · B.m3 · A.m4

Static = **hidden**, chosen by the reference type; instance = **overridden**, chosen by the object.

21 Code questions: ++, <<, defaults, lambdas

Q48: Write the operator overloading functions in C++ for the class `Integer { private: int i; }` so that `main() { Integer P; P++; ++P; }` works.

✎ **Asked in:** BUET AP 2025

```
class Integer {
    int i;
public:
    Integer(int v = 0) { i = v; }
    Integer& operator++() { ++i; return *this; }      // prefix ++P
    Integer operator++(int) {                          // postfix P++
        Integer old = *this; ++i; return old; }        // dummy int
    int get() const { return i; }
};
Integer P;  Integer a = P++;  Integer b = ++P;
```

→ a = 0, b = 2, P = 2

The **dummy int** parameter only tells the compiler "this is postfix"; postfix returns the **old** value by copy.

Q49: What is a functional interface in Java? Give an example.

✎ **Asked in:** BUET M.Sc. April 2024

```
@FunctionalInterface
interface Square { int calc(int x); }     // exactly one abstract method
Square s = x -> x * x;                    // a lambda supplies it
System.out.println(s.calc(7));
```

→ 49

An interface with **exactly one abstract method** (default/static methods allowed), so a lambda can implement it: `Runnable`, `Comparator`.

MCQ as asked	answer	asked in
`interface Alpha { default void reset(){} }, interface Beta { default void reset(){} }, class Gamma implements Beta, Alpha {}`: which fixes the error?	**interface Beta extends Alpha**	BUET M.Sc. April 2021
In `class G<A extends X & Y & Z>`: (a) X, Y classes, Z interface (b) X, Y interfaces, Z class (c) all interfaces (d) all classes	**(c)**	BUET M.Sc. April 2021

Two unrelated defaults clash ("types Beta and Alpha are incompatible"); if `Beta extends Alpha`, Beta's version wins. In a bound only the **first** type may be a class; the rest must be interfaces, so of the options only (c) compiles.

Exam trap: `void f() = 0;` without `virtual` is a compile error; and `static` methods are never virtual.

22 Constructor and destructor

Q50: What is a constructor function? Write its properties. / Write an example of a constructor. / When is a constructor invoked in OOP?

✎ **Asked in:** WZPGCL AE(CSE) 2023 · WZPDCL AE(CSE) 2019 · BPSC AP 2022

Constructor: a special member function with the **same name as the class** that runs **automatically when an object is created** and initialises it. It is **invoked** at every object creation: a declaration `Point p;`, `new Point()`, a copy, or passing an object by value.

- ✓ same name as the class
- ✓ **no return type**, not even `void`
- ✓ called automatically, once per object
- ✓ can be **overloaded**; may take default arguments
- ✓ not inherited; cannot be `virtual` (C++) or `static`/`final`/`abstract` (Java)
- ✓ if you write none, the compiler gives a default one

```
class Point {
    int x, y;
public:
    Point() { x = y = 0; }                 // default
    Point(int a, int b) { x = a; y = b; } // parameterised
    void show() { cout << x << "," << y << " "; }
};
Point p;  Point q(3, 4);  p.show();  q.show();
```

→ 0,0 3,4 (kinds: default · parameterised · copy)

Q51: Why are constructor and destructor functions used in OOP? Give examples of each in C++ or Java. / Main difference between destructor and constructor, with an example. / What are class and object? Difference between constructors and destructors. / Define constructor, destructor and copy constructor.

✎ **Asked in:** BPSC SAE 2021 · PSB Programmer 2018 · 17th NTRCA CSE (2023) · 47th BCS ICT

	Constructor	Destructor
job	**initialise** the object, acquire resources	**clean up**, release memory / files
name	`Point()`	`~Point()` (tilde)
runs	when the object is created	when it goes out of scope / `delete`
arguments	may take; may be overloaded	**none**; exactly one per class
order with inheritance	base → derived	derived → base

23 Destructor at work, copy constructor

Constructor + destructor in one run (for the cards on the previous sheet):

```
class Logger {
    string name;
public:
    Logger(string n) : name(n) { cout << "open " << name << "\n"; }
    ~Logger() { cout << "close " << name << "\n"; }
};
int main() {
    Logger a("A");
    { Logger b("B"); }          // b dies at this brace
    cout << "end of main\n";
}
```

→ open A · open B · close B · end of main · close A

A **copy constructor** `Point(const Point &p)` builds a new object from an existing one (below). Java has **no destructor**: the garbage collector frees memory; use `try-with-resources` for files.

Q52: Write a program in C++ using a copy constructor to initialise its member variables from another object.

✎ **Asked in:** BUET M.Sc. May 2016

```
class Student {
    string name; int roll;
public:
    Student(string n, int r) { name = n; roll = r; }
```

→ copy constructor called
→ Rafi 7
→ copy constructor called
→ Rafi 7

```
    Student(const Student &s) {              // copy constructor
        name = s.name; roll = s.roll;
        cout << "copy constructor called\n"; }
    void show() { cout << name << " " << roll << "\n"; }
};
void print(Student s) { s.show(); }          // by value
int main() {
    Student s1("Rafi", 7);
    Student s2 = s1;                         // from another object
    s2.show();   print(s1);
}
```

The parameter is a **reference**: `Student(Student s)` would need a copy to make the copy — infinite recursion, so the compiler rejects it.

MCQ If an object is passed by value: (a) copy constructor copies the values into a temporary object (b) copy constructor copies the values into another object in the function (c) a reference to the object is used (d) a reference creates a new object in its place → **(b)**: the second "copy constructor called" above is the parameter `s` inside `print`. ✎ BUET IICT April 2021

MCQ `class A { int a; public: A(int a) { this->a = a; } }; class B : A { int b; public: ___ };` A valid constructor for B? (a) `B(int a, int b) : A(a), b(b) { }` (b) `B(int a, int b) { super(a); this->b = b; }` (c) `B(int a, int b) : A(a), this->b(b) { }` (d) none → **(a)**; (b) is Java syntax and A has no default constructor, (c) is not a valid initialiser. ✎ BUET M.Sc. April 2021

24 Default constructor, `static`, objects in a list

Q53: C++ code given: write what is necessary in the class for the code to compile (a constructor).
✎ **Asked in:** BUET M.Sc. October 2018

The usual case: the class has only `Point(int a, int b)`, and `main` writes `Point p;`. Error "no matching constructor for initialization of 'Point'": once you write any constructor, the free default one disappears. Add **Point() {}** or give defaults: `Point(int a = 0, int b = 0)`.

Q54: Write a method to find out how many times an object has been instantiated in Java. / Write a program using a static variable and a static method to explain `static`. / What syntax is used for calling static methods in a class?
✎ **Asked in:** BUET M.Sc. May 2016 · 15th NTRCA ICT · BCC AP 2023

```
class Item {
    private static int count = 0;      // one copy for all objects
    Item() { count++; }                // every constructor call counts
    static int getCount() { return count; }
}
new Item(); new Item(); new Item();
System.out.println(Item.getCount());   // ClassName.method()
```

→ 3

A **static** member belongs to the **class**, not to an object: one shared copy, usable without any object. Call it as `ClassName.method()` (Java) or `ClassName::method()` (C++). A static method cannot use `this`.

Q55: Write a Java class with a constructor, setter and getter; create 7 objects of it and put them in an ArrayList.
✎ **Asked in:** BPDB AE(CSE) 2018

```
class Book {
    private String title;  private int price;
    Book(String title, int price) { this.title = title; this.price = price; }
    String getTitle() { return title; }   int getPrice() { return price; }
    void setPrice(int price) { this.price = price; }
}
ArrayList<Book> list = new ArrayList<>();
```

→ size 7 · first price 150 · last Book7

```
for (int i = 1; i <= 7; i++) list.add(new Book("Book" + i, 100 * i));
list.get(0).setPrice(150);
```

MCQ as asked	answer	asked in
Which does not return any value? constructor / friend / inline / member function	**constructor**	BPSC AP 2022
A constructor is a special type of: class / field / method / property	**method**	BPSC AME 2019
What is invoked to create an object in Java?	**a constructor**	BPDB AE(CSE) 2021

25 C++ vs Java, processors, data types

Q56: What are the differences between the C++ and Java programming languages?
✎ **Asked in:** 36th BCS (2016)

Point	C++	Java
runs as	native machine code	**bytecode on the JVM**
platform	recompile per platform	write once, run anywhere
pointers	yes	no (references only)
memory	`new`/`delete`, destructors	**garbage collector**
multiple inheritance	of classes	only through interfaces
operator overloading	yes	no
generic code	templates	generics
global functions	allowed	everything inside a class

Q57: Define the type of language processor for Python, C, C++ and Java.
✎ **Asked in:** BUET M.Sc. May 2016

C, C++: **compiler** → native machine code. **Java**: compiler (`javac` → bytecode) **and** the JVM, which interprets and JIT-compiles it: a **hybrid**. **Python**: **interpreter** (CPython first compiles to bytecode, then interprets it).

Q58: Describe the different data types in the C++ programming language.
✎ **Asked in:** 13th NTRCA CSE (2016)

group	types (size on a 64-bit compiler, printed by `sizeof`)
built-in	`char` 1 · `bool` 1 · `short` 2 · `int` 4 · `long long` 8 · `float` 4 · `double` 8 · `void`
derived	array, pointer, reference, function
user-defined	`class`, `struct`, `union`, `enum`, `typedef`/`using`

Q59: Rewrite a Java Swing code using lambda expressions.
✎ **Asked in:** BUET M.Sc. October 2017

```
b.addActionListener(new ActionListener() {                    // before: anonymous class
    public void actionPerformed(ActionEvent e) { System.out.println("clicked"); }
});
b.addActionListener(e -> System.out.println("clicked"));      // after: lambda
```

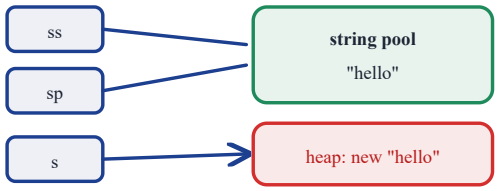
`ActionListener` has one abstract method, so it is a functional interface and a lambda can replace the anonymous class (both compile; the button then has 2 listeners).

> **Memory trick:** Java = "**C++ minus**": minus pointers, minus `delete`, minus operator overloading, minus multiple class inheritance.

26 Find the output: strings, chars, assignments

> **Q60:** Find the output: `String s = new String("hello"); String ss = "hello"; String sp = "hello"; print(s.compareTo(ss)); print(s == ss); print(s == sp); print(ss == sp);` / Given Java code using strings, what is the output?
> ✎ **Asked in:** BUET M.Sc. May 2019

→ 0 (same characters)
→ false
→ false
→ true
(and `s.equals(ss)` → true)



`==` compares **references**; `equals()` / `compareTo()` compare **contents**. Literals share one object in the **string pool**; `new String` makes a new one. (BUET's second 2019 string question, not preserved, tests the same rule.)

MCQ as asked	answer	asked in
`System.out.print('D' + 'E' + 'F');` (a) 137 (b) DEF (c) 207 (d) D E F	**(c) 207**	Combined Bank AHE/AE(IT) 2021
`int a=10, b=15; if((a=100)==(b=a)) print(b+a); else print(b-a);` (a) 100 (b) 200 (c) 0 (d) 3	**(b) 200**	Combined Bank AHE/AE(IT) 2021
Valid ArrayList of String: (a) `List<String> list = new ArrayList<>();` (b) `List<>` … (c), (d) `new List<>()`	**(a)**	BUET M.Sc. April 2021

`char + char` is **int** addition: 68 + 69 + 70 = **207**; `"" + 'D' + 'E' + 'F'` would print DEF. In the second, `a` becomes 100, then `b = a` = 100, so the test is true and `b + a` = 200.

> **Exam trap:** The printed key says (a) 100; the program prints **200** (run).

> **Q61:** Java code given: find the length of each string in a string list without using a for loop.
> ✎ **Asked in:** BUET M.Sc. October 2018

```
List<String> list = List.of("java", "c", "python");
list.stream().map(String::length).forEach(System.out::println);
list.forEach(s -> System.out.println(s + " " + s.length()));
```

→ 4 · 1 · 6
→ java 4 · c 1 · python 6

> **Q62:** A Java program: find its output (the program was not preserved in any copy).
> ✎ **Asked in:** WZPDCL AE(CSE) 2019

Practise with the four outputs on this sheet.

27 Short programs asked in OOP papers

Q63: Write a program in any programming language that reads five numbers from the keyboard and displays the smallest, the largest and the average of those numbers.

✎ **Asked in:** BDB SO(IT) 2017

```
// inside main(); import java.util.Scanner
Scanner sc = new Scanner(System.in);
double min = Double.MAX_VALUE, max = -Double.MAX_VALUE, sum = 0;
for (int i = 0; i < 5; i++) {
    double x = sc.nextDouble();
    sum += x;  if (x < min) min = x;  if (x > max) max = x;
}
System.out.println("Smallest " + min + ", largest " + max
                   + ", average " + sum / 5);
```

input 4 9 -2 7.5 1 → Smallest -2.0, largest 9.0, average 3.9

Exam trap: Start `max` at `-Double.MAX_VALUE`, not `Double.MIN_VALUE`: MIN_VALUE is the smallest **positive** double, so all-negative input would print a wrong maximum.

Q64: Write a program in C++ to check whether a given number is prime or not.

✎ **Asked in:** 47th BCS ICT

```
#include <iostream>
using namespace std;
bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)        // up to √n is enough
        if (n % i == 0) return false;
    return true;
}
int main() {
    int n;  cin >> n;
    cout << n << (isPrime(n) ? " is prime" : " is not prime");
}
```

97 → 97 is prime
91 → 91 is not prime (7 × 13)

If `n` has a factor bigger than √**n**, it also has one smaller, so the loop stops at `i*i <= n`.

Q65: Write a Java program that takes three inputs and finds the factors of each.

✎ **Asked in:** NHA AP 2019

```
static void factors(int n) {
    System.out.print("Factors of " + n + ":");
    for (int i = 1; i <= n; i++) if (n % i == 0) System.out.print(" " + i);
    System.out.println();
}
// main(): Scanner sc = new Scanner(System.in);
//         for (int k = 0; k < 3; k++) factors(sc.nextInt());
```

input 12 7 20
→ Factors of 12: 1 2 3 4 6 12
→ Factors of 7: 1 7
→ Factors of 20: 1 2 4 5 10 20

Q66: "What is the output of the code snippet?" / "C++ program" (the code or task was not preserved in any copy).

✎ **Asked in:** BCIC AP 2022 · DUET AP 2020

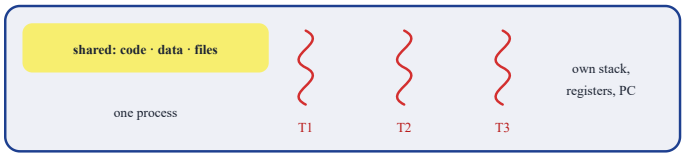
Practise the traced programs on sheets 13, 20, 24 and 27: constructor order, virtual calls, copy constructors, `==`.

28 Thread, multithreading, multitasking

Q67: Define a thread. / What is multithreaded programming? Why is multithreading used in programming?
✎ **Asked in:** 15th NTRCA · Combined Bank AP 2024

Thread: the smallest unit of execution scheduled by the OS: a single sequence of instructions with its **own stack, registers and program counter**, sharing the code, data and open files of its **process** with the other threads.

Multithreading = one program runs several threads at once (truly parallel on several cores, interleaved on one). **Why:** ① responsive UI while a download runs · ② speed on multi-core CPUs · ③ threads share memory, cheaper than processes · ④ servers handle many clients. **Cost:** shared data needs **synchronisation**, or race conditions appear (see the overdraft account, next sheet).



three threads inside one process

Q68: Write down the difference between multitasking and multiprocessing.
✎ **Asked in:** DESCO SAE 2023

	Multitasking	Multiprocessing
meaning	several tasks share **one CPU** by time-slicing	several **CPUs / cores** run tasks truly in parallel
parallelism	apparent (fast switching)	real
hardware	one processor is enough	needs 2 or more processors
goal	responsiveness, many users/apps	throughput, speed

Q69: Thread in C++: calculate a factorial (not with pthread).
✎ **Asked in:** BUET M.Sc. October 2017

```
#include <iostream>
#include <thread>
using namespace std;
void factorial(int n, unsigned long long &result) {
    result = 1;
    for (int i = 2; i <= n; i++) result *= i;
}
int main() {
    unsigned long long r1, r2;
    thread t1(factorial, 10, ref(r1));     // std::thread
    thread t2(factorial, 20, ref(r2));
    t1.join();  t2.join();                 // wait for both
    cout << "10! = " << r1 << "\n20! = " << r2;
}
```

→ 10! = 3628800
→ 20! = 2432902008176640000

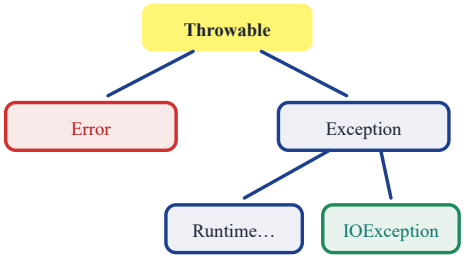
`std::ref` is needed to pass a reference into a thread; `join()` waits. 20! is the largest factorial that fits in 64 bits.

29 Exceptions, and a thread-safe account

Q70: What is the difference between an exception and an error in Java? / What is exception handling? Write with an example.

✎ **Asked in:** SPCBL SAE 2022

Exception	Error
the program **can handle** it	serious JVM / system fault
recover with `try-catch`	should **not** be caught
checked or unchecked	always unchecked
`FileNotFoundException`	`OutOfMemoryError`, `StackOverflowError`



Exception handling catches a run-time problem so the program does not crash: `try` (code that may fail) → `catch` (handle it) → `finally` (always runs: close files) · `throw` raises one · `throws` declares one.

```
int[] a = {10, 20};
try {
    System.out.println(a[0] / 0);
} catch (ArithmeticException e) {
    System.out.println("Caught: " + e.getMessage());
} finally {
    System.out.println("finally always runs");
}
System.out.println("program continues");
```

→ Caught: / by zero
→ finally always runs
→ program continues

Q71: Implement an "Overdraft Account (OD)" class in Java: opened with an approved loan limit (e.g. 100000); deposit any amount at any time; draw beyond the balance while the overdraw stays within the limit; the code runs in multithreading mode. (The rest is cut off in the source.)

✎ **Asked in:** BB AP 2019

```
class ODLimitExceeded extends Exception { ODLimitExceeded(String msg) { super(msg); } }
class ODAccount {
    private double balance = 0;  private final double limit;
    ODAccount(double limit) { this.limit = limit; }
    synchronized void deposit(double amt) { balance += amt; }
    synchronized void draw(double amt) throws ODLimitExceeded {
        if (balance - amt < -limit) throw new ODLimitExceeded("limit " + limit + " exceeded");
        balance -= amt; }
    synchronized double getBalance() { return balance; }
}
ODAccount acc = new ODAccount(100000);
Runnable job = () -> { for (int i = 0; i < 1000; i++) acc.deposit(10); };
Thread t1 = new Thread(job), t2 = new Thread(job);  t1.start(); t2.start(); t1.join(); t2.join();
try { acc.draw(120000); System.out.println(acc.getBalance()); acc.draw(1); }
catch (ODLimitExceeded e) { System.out.println("Refused: " + e.getMessage()); }
```

→ 20000.0 (always: `synchronized`) · -100000.0 · Refused: limit 100000.0 exceeded

30 Facade pattern, and modelling a company

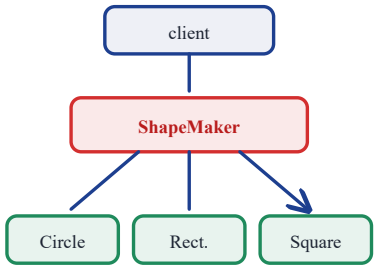
Q72: A `Shape` interface with `Circle`, `Rectangle`, `Square`; a `ShapeMaker` class and `FacadePatternDemo`: name the pattern and implement it.

✎ **Asked in:** BPDB AE(CSE) 2023

It is the **Facade pattern** (structural): one simple class (`ShapeMaker`) in front of a set of classes, so the client calls `drawCircle()` without knowing which objects do the work.

```
interface Shape { void draw(); }
class Circle implements Shape {
    public void draw() { System.out.println("Circle::draw()"); } }
class Rectangle implements Shape {
    public void draw() { System.out.println("Rectangle::draw()"); } }
class Square implements Shape {
    public void draw() { System.out.println("Square::draw()"); } }
class ShapeMaker {                                  // the facade
    private Shape circle = new Circle(), rectangle = new Rectangle(),
                  square = new Square();
    public void drawCircle()    { circle.draw(); }
    public void drawRectangle() { rectangle.draw(); }
    public void drawSquare()    { square.draw(); }
}
public class FacadePatternDemo {
    public static void main(String[] args) {
        ShapeMaker m = new ShapeMaker();
        m.drawCircle(); m.drawRectangle(); m.drawSquare();
    }
}
```

→ Circle::draw()
→ Rectangle::draw()
→ Square::draw()



Q73: Write a class to keep the data of employees, projects, and the employee information of a company.

✎ **Asked in:** BUET M.Sc. October 2018

```
class Employee { int id; String name;
    Employee(int id, String name) { this.id = id; this.name = name; } }
class Project { String code; List<Employee> members = new ArrayList<>();
    Project(String code) { this.code = code; }
    void assign(Employee e) { members.add(e); } }
class Company { List<Employee> staff = new ArrayList<>();
    List<Project> projects = new ArrayList<>();
    void hire(Employee e) { staff.add(e); }
    void info(Employee e) {                    // the projects of one employee
        StringBuilder sb = new StringBuilder(e.name + ":");
        for (Project p : projects) if (p.members.contains(e)) sb.append(" ").append(p.code);
        System.out.println(sb); } }
```

Sami on ERP + Web, Lina on Web
→ Sami: ERP Web
→ Lina: Web

Company **has** employees and projects (**composition**, HAS-A, not IS-A); employees ↔ projects is many-to-many.

★ Night before the exam

1. **Pillars: A PIE** — abstraction, polymorphism, inheritance, encapsulation; "three principles" = E, I, P.
2. **Overloading** = same name, other parameters, compile time · **overriding** = same signature in a subclass, run time (8 exams).
3. **Friend function**: non-member with access to private data; no `this`; breaks encapsulation.
4. **Inheritance**: single, multilevel, hierarchical, multiple, hybrid; diamond → `virtual` base. Constructors A → B → C, destructors reverse.
5. **Constructor**: class name, no return type, auto-called, overloadable. Destructor: `~`, no arguments, one per class. Copy constructor takes `const T&`.
6. **Access**: public YYYY · protected YYYN · private YNNN; C++ class default private, struct public.
7. **Virtual**: late binding by the object; pure virtual `= 0` → abstract class; Java `final` stops overriding.
8. **Java traps**: `'D'+'E'+'F'` = 207 · `==` on strings compares references · `(a=100)==(b=a)` → 200.

✓ Practice MCQs (all asked; answers at the bottom)

1. Which declares a pure virtual function? (a) `virtual void f();` (b) `virtual void f() = 0;` (c) `void f() = 0;` (d) `pure virtual f();` ✎ 49th BCS ICT (2025)

2. Compile-time polymorphism: (a) virtual function (b) operator overloading (c) overriding (d) dynamic binding ✎ 49th BCS CSE (2025)

3. Default inheritance in C++: (a) public (b) private (c) protected (d) error ✎ BPSC AME 2019

4. Constructor order for C : B : A — (a) C B A (b) A C B (c) A B C (d) C A B ✎ BUET IICT April 2021

5. Prevents overriding in Java: (a) const (b) final (c) static (d) synchronized ✎ 49th BCS ICT (2025)

6. `System.out.print('D'+'E'+'F');` (a) 137 (b) DEF (c) 207 (d) D E F ✎ Combined Bank AHE/AE(IT) 2021

7. Does NOT achieve encapsulation: (a) private (b) classes (c) getters/setters (d) global variables ✎ BB AD(ICT) 2025

8. Does not return any value: (a) constructor (b) friend (c) inline (d) member function ✎ BPSC AP 2022

Answers: 1 b · 2 b · 3 b · 4 c · 5 b · 6 c · 7 d · 8 a

Memory trick: Before writing any OOP answer: **define → table → code → output → trap**. Five boxes, full marks.

Exam trap: Never mix C++ and Java in one program: `this.x`, `new` without a pointer, or `extends` in C++ cost marks.